FILE Handling

What is File handling?

→ App reading data from file, writing data into the file is known as file handling

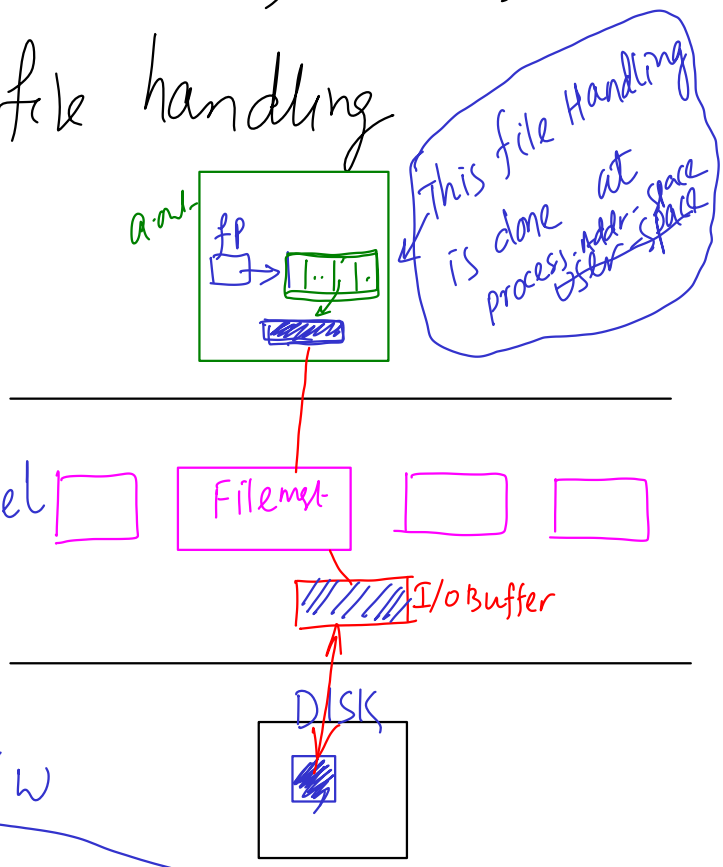
```
fp = fopen("f1.txt", "r");
```

app

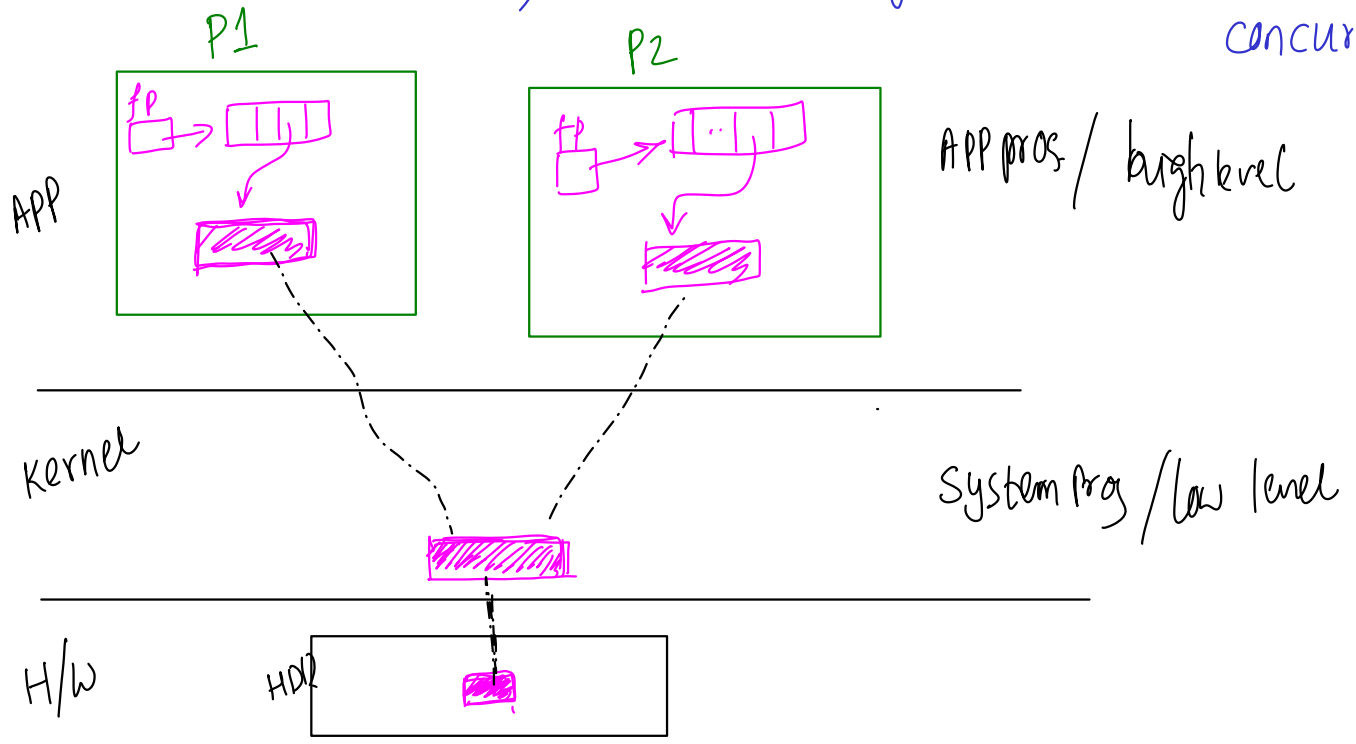
≡

```
fread(    );
```

⇒



Lets assume a scenario, where two (more than one) processes trying to Access the same file, concurrently.



Low level File Handling

- → Accessing File using File handling System call
- → Access the file in kernel space
- → Accessing file via FileDescripter NOT File-Pointer

Why we need low level file handling?

- → To have Greater control on file Access. ✓
- → Less Memory Usage / faster Access ✓
- → To Access Spl files.

$\boxed{\text{open()}}$ read() / write() close()

```
int open( const char *  ,  int  );
          Filename        flags
```

- → -1 on failure
- → on Success, lowest unused file Descripter

flags:
- → O_RDONLY
- → O_WRONLY
- → O_RDWR

fcntl()

ioctl()

etc..

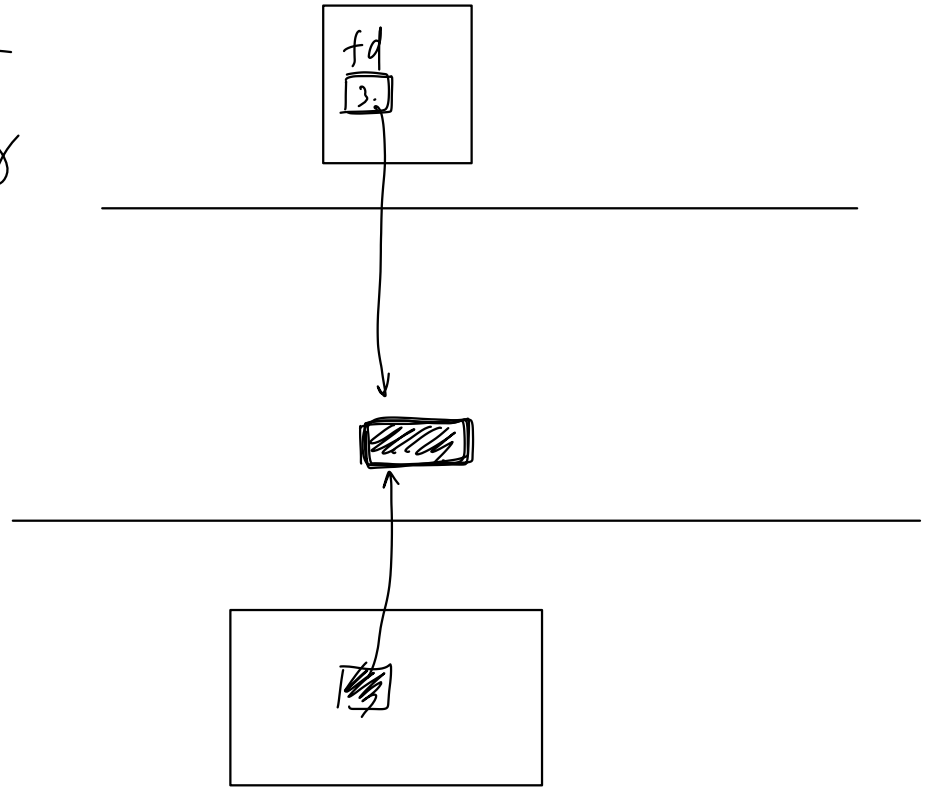
file-Descriptor

=> low-level file handler.

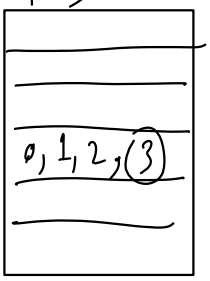
=> A non-negative Small integer.

=> By-default, there're three file-descriptors open, in-every process, they're

0 => stdin
1 => stdout
2 => stderr



File Descriptor Entries present in PCB.



Using open() successfully, an entry is made available in.

open File table - of PCB, reflecting in the program,

by the file descriptor

a.out (1769)

```
fd = open(-,-);
```

PCB

1769
0, 1, 2, 3

```
fd = open("f2", O_WRONLY);
```

≡

```
write(fd, "VECTOR", 6);
```

⇒ "f2" named file Doesn't exist

→ fails, returns -1

=

⇒ "f2" named file is existing,

then open is success, writing is permitted

f2

ABCDEFGH

↓

VECTORGH